SecVote: A Privacy-Preserving Blockchain-Based Electronic Voting System using Merkle Proofs and Nullifiers

A PROJECT REPORT

Submitted by

Suhas Sudheer Upadhya[RA2311026010897]

Adhithya Raghunathan[RA2311026010880]

Under the Guidance of

Grace Shalini T

Assistant professor, Department of Computing Technologies

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

[with specialization in ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR - 603 203

NOVEMBER 2026

Declaration Form

Degree / Course	: BTECH / CSE AMIL
Student Name	: Suhas Upadhya , Adhithya Ragunathan
Registration Number	: RA2311026010897 , RA2311026010880
Title of Work	: SecVote: A Privacy-Preserving Blockchain-Based Electronic Voting System using Merkle Proofs and Nullifiers

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that we have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc.)
- Given the sources of all pictures, data etc. that are not our own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Complied with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Suhas Upadhya [RA2311026010897]

Adhithya Raghunathan [RA2311026010880]

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603203

BONAFIDE CERTIFICATE

Certified that [21CSC402P] - Project report titled **"SECVOTE:A PRIVACY-PRESERVING BLOCKCHAIN-BASED ELECTRONIC VOTING SYSTEM USING MERKLE PROOFS AND NULLIFIERS"** is the bonafide work of [Suhas Upadhya[RA2311026010897],Adithya Raghunathan[RA2311026010880]] who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Grace Shalini T
Assistant Professor
Department of Computing Technologies
SRM Institute of Science and Technology
Kattankulathur

SIGNATURE
Dr. Alice Nithya
Professor and Head of
Department of Computing Technologies
SRM Institute of Science and Technology
Kattankulathur

ACKNOWLEDGEMENTS

We express our humble gratitude to Dr. C. Muthamizhchelvan, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to Dr. Leenus Jesu Martin M, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank Dr. Revathi Venkataraman, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We are incredibly grateful to our Head of the Department, Dr. Alice Nithya, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators Grace Shalini T, Department of Computing Technologies, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, [Aarthy K], Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

We sincerely thank all the staff and students of the Department of Computing Technologies, School of Computing, SRM Institute of Science and Technology, for their help during our project, including everyone who helped us test the system during the mock elections. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

ABSTRACT

Electronic voting promises convenience and speed, but it forces a difficult trade-off between two properties that pull in opposite directions: voter anonymity and election verifiability. Centralised voting portals ask voters to trust a single server not to alter tallies or to record who voted for whom, while naive blockchain voting exposes wallet addresses, imposes gas costs and requires voters to install and fund crypto wallets.

This project presents SecVote, a blockchain-based electronic voting platform designed for campus and institutional elections. Eligibility is proven with Merkle proofs: only the 32-byte Merkle root of the eligible voter list is stored on an Ethereum Virtual Machine (EVM) smart contract, so no personal information is published on-chain. Identity is established off-chain through a voter application with ID-proof PDF upload and administrator review, and each ballot is protected by two separate credentials: a secret voting key and a six-digit e-mail one-time password delivered through Gmail SMTP with a Resend API fallback. A relational audit log recorded 338 events, covering OTP requests, proof generation and ballot relays, during testing. The system was verified with 19 automated tests (five contract tests, six OTP tests and eight end-to-end workflow tests), all of which passed, and was exercised in mock campus elections.

The results show that Merkle-proof eligibility and nullifier-based replay protection can be combined with a gasless relayer to give a usable, publicly auditable voting workflow. The report also documents the limitations of the current design, chiefly the trust placed in the off-chain identity authority and the relayer, and outlines zero-knowledge membership proofs and decentralised relaying as future work.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
ABSTRACT		**6**
TABLE OF CONTENTS		**7**
LIST OF FIGURES		**9**
LIST OF TABLES		**10**
ABBREVIATIONS		**11**
1	**INTRODUCTION**	**1**
	1.1 Introduction	1
	1.2 Problem Statement	2
	1.3 Motivation	2
	1.4 Sustainable Development Goals of the Project	3
	1.5 Organisation of the Report	3
2	**LITERATURE SURVEY**	**4**
	2.1 Limitations Identified from Literature Survey (Research Gaps)	5
	2.2 Research Objectives	6
	2.3 Product Backlog (Key User Stories with Desired Outcomes)	6
	2.4 Plan of Action (Project Road Map)	8
3	**SPRINT PLANNING AND EXECUTION METHODOLOGY**	**9**
	3.1 SPRINT 1	9
	3.1.1 Objectives with User Stories of Sprint 1	9
	3.1.2 Functional Document	9
	3.1.3 Architecture Document	11
	3.1.4 Outcome of Objectives / Result Analysis	13
	3.1.5 Sprint Retrospective	13
	3.2 SPRINT 2	14
	3.2.1 Objectives with User Stories of Sprint 2	14
	3.2.2 Functional Document	14
	3.2.3 Architecture Document	16
	3.2.4 Outcome of Objectives / Result Analysis	17
	3.2.5 Sprint Retrospective	18
	3.3 SPRINT 3	18
	3.3.1 Objectives with User Stories of Sprint 3	18

3.3.2 Functional Document 18
3.3.3 Architecture Document 20
3.3.4 Outcome of Objectives / Result Analysis 21
3.3.5 Sprint Retrospective 22
4 RESULTS AND DISCUSSIONS 23
4.1 Project Outcomes (Performance Evaluation, Comparisons, Testing Results) 23
4.1.1 Verification Results 23
4.1.2 On-Chain Cost 25
4.1.3 Proof Size and Scalability 25
4.1.4 Operational Evidence 26
4.2 Threat Analysis 26
4.2.2 Limitations 27
4.2.3 Comparison with Related Work 28
5 CONCLUSION AND FUTURE ENHANCEMENT 31
5.1 Conclusion 31
5.2 Future Enhancement 31
REFERENCES 33
APPENDIX 34
A CODING 34

ABBREVIATIONS

Abbreviation	Expansion
ABI	Application Binary Interface
API	Application Programming Interface
DB	Database
E2E	End-to-End
eKYC	Electronic Know Your Customer
EVM	Ethereum Virtual Machine
HSM	Hardware Security Module
JWT	JSON Web Token
ORM	Object-Relational Mapping
OTP	One-Time Password
PII	Personally Identifiable Information
REST	Representational State Transfer
RPC	Remote Procedure Call
SDG	Sustainable Development Goal
SMTP	Simple Mail Transfer Protocol
SSL / TLS	Secure Sockets Layer / Transport Layer Security
TTL	Time To Live
UI / UX	User Interface / User Experience
ZKP	Zero-Knowledge Proof

CHAPTER 1
INTRODUCTION

1.1 Introduction

Voting is the mechanism through which an institution turns the individual preferences of its members into a collective decision, and the legitimacy of that decision rests on trust in the process. Paper ballots earn this trust through physical observation, but they are slow to count, expensive to administer and difficult to run for remote participants. Digital voting removes much of this friction, yet it introduces a new problem: the voter can no longer watch the ballot box.

Two properties are central to any credible election. The first is **ballot secrecy**: nobody, including the administrators, should be able to link a particular ballot to the person who cast it. The second is **verifiability**: anyone should be able to check that only eligible people voted, that each person voted at most once, and that the published tally matches the ballots cast. In a conventional web portal these properties conflict, because the server that knows who is allowed to vote is also the server that stores the votes and reports the result.

Blockchain technology offers a way to loosen this conflict. A smart contract running on an Ethereum Virtual Machine (EVM) network [8] behaves as a public, append-only bulletin board whose rules are enforced by code rather than by an administrator. However, putting an entire voting system on a public ledger creates its own difficulties: every voter would need a wallet and funds for gas, and publishing voter identities or addresses next to their choices would destroy ballot secrecy.

SecVote is a voting platform built to address these difficulties for campus and institutional elections. It combines three ideas. First, eligibility is checked with a **Merkle proof**, so the chain stores only a single 32-byte root and never a list of voters. Second, double voting is stopped with a **nullifier**, a hash derived from each voter's secret that the contract records the first time it is used and refuses afterwards. Third, a backend **relayer** submits transactions and pays the gas, so a voter only needs a browser. Identity proofing, e-mail one-time passwords and an audit trail are layered around this core to make the system usable in a real election.

1.2 Problem Statement

Conventional electronic voting machines and web portals require the voter to trust a central server or database administrator not to alter tallies, not to log voter identities alongside their choices, and not to inject ballots that were never cast. The voter has no independent way to check any of these things after the fact.

Naive blockchain voting replaces that trust with a different set of problems. Ballots and wallet addresses are exposed on a public ledger, gas costs grow with the amount of data processed on-chain, and voters must install a wallet extension such as MetaMask and obtain cryptocurrency before they can take part. For a student council or faculty election these requirements are unrealistic. There is therefore a need for a voting system that keeps voter identity off the ledger, enforces one-person-one-vote with a mathematical guarantee, remains publicly auditable, and does not ask voters to handle cryptocurrency.

1.3 Motivation

The motivation for this project comes from the gap between what the voting literature proposes and what a typical institution can actually deploy. Many published schemes assume voters who are comfortable generating keys, signing transactions and paying fees, or they depend on heavy cryptographic protocols that need several interactive rounds. A student voting from a phone between classes will not tolerate that, and an election officer cannot support it.

A second motivation is the practical observation that most institutional elections already have a trusted registry of who is entitled to vote, such as a student roll or a membership list.

Finally, the project was motivated by engineering curiosity about the cost structure of on-chain systems. Storing one record per voter on-chain costs gas in proportion to the number of voters, whereas committing to the whole list with a Merkle root costs a constant amount. Demonstrating this trade-off in a working system, with measurements and automated tests, was a deliberate goal.

1.4 Sustainable Development Goals of the Project

SecVote contributes to the following United Nations Sustainable Development Goals [10].

- **SDG 16 - Peace, Justice and Strong Institutions.** Target 16.6 calls for effective, accountable and transparent institutions at all levels, which SecVote supports through tamper-evident, publicly verifiable tallies recorded on a smart contract. Target 16.7 calls for responsive, inclusive, participatory and representative decision-making, which SecVote supports by allowing eligible members to vote remotely and securely.
- **SDG 9 - Industry, Innovation and Infrastructure.** Target 9.c concerns widening access to information and communications technology. By removing the need for wallets and cryptocurrency, SecVote makes a blockchain-backed service reachable from an ordinary browser.

1.5 Organisation of the Report

Chapter 2 surveys related work, derives the research gaps and objectives, and presents the product backlog and plan of action. Chapter 3 describes the three sprints used to build the system, each with its user stories, functional document, architecture document, outcomes and retrospective. Chapter 4 reports the results of the verification tests, the on-chain cost analysis and the observed limitations. Chapter 5 concludes the report and suggests future enhancements. The references and an appendix listing the key code artifacts follow.

CHAPTER 2
LITERATURE SURVEY

The papers below [1]-[7] were reviewed to understand how earlier work balances privacy, verifiability and usability in blockchain-based voting. Foundational cryptographic references on verifiable elections and hash trees are included because SecVote relies directly on them.

Table 2.1 Literature survey

S.No	Title	Journal and Author(s)	Methodology (Key Findings)	Gaps and Limitations
1	Verifiable Secret-Ballot Elections	J. Benaloh, Ph.D. thesis, Yale University (1987)	Introduced the formal requirements of verifiable secret-ballot elections, including universal verifiability, and a cryptographic scheme meeting them.	Cryptographic protocol with heavy computation; not packaged as a deployable web system for institutions.
2	A Digital Signature Based on a Conventional Encryption Function	R. C. Merkle, CRYPTO '87 (1987)	Described the hash tree in which a single root commits to many leaves and membership is shown with a logarithmic-size path.	General-purpose construction; does not address voting, identity or replay protection.
3	Blockchain-Enabled E-Voting	N. Kshetri and J. Voas, IEEE Software 35(4), 95-99 (2018)	Surveyed how blockchain e-voting could reduce voter fraud and widen access, with deployments and challenges discussed.	Conceptual overview; offers no implementation, cost measurements or test evidence.
4	A Smart Contract for Boardroom Voting with Maximum Voter Privacy	P. McCorry, S. F. Shahandashti and F. Hao, Financial Cryptography and Data Security, LNCS 10322, 357-375 (2017)	Implemented a self-tallying, privacy-preserving vote on Ethereum for small groups, with the tally computed by the contract.	Aimed at small groups; voters transact from their own accounts and cryptographic work is done on-chain.
5	Blockchain-Based E-Voting System	F. Þ. Hjálmarsson, G. K. Hreiðarsson, M. Hamdaqa and G. Hjálmtýsson, IEEE CLOUD 2018, 983-986	Compared blockchain frameworks and proposed a permissioned-blockchain voting system, discussing security and legal limits for national elections.	Depends on trusted parties for registration; authors note limitations for large national-scale use.
6	End-to-End Voting with Non-Permissioned and Permissioned Ledgers	S. Bistarelli, I. Mercanti, P. Santancini and F. Santini, J. Grid Computing 17(1), 97-118 (2019)	Built end-to-end voting on Bitcoin and on the MultiChain permissioned ledger; the permissioned variant adds uncoercibility and receipt-freeness and avoids Bitcoin costs.	Bitcoin variant lacks some privacy properties; platform differs from EVM smart-contract ecosystems.

S.No	Title	Journal and Author(s)	Methodology (Key Findings)	Gaps and Limitations
7	E-Voting with Blockchain: An E-Voting Protocol with Decentralisation and Voter Privacy	F. Hardwick, A. Gioulis, R. N. Akram and K. Markantonakis, IEEE iThings/GreenCom /CPSCom/SmartData 2018 (arXiv:1805.10258)	Proposed a blockchain voting protocol emphasising decentralisation and voter privacy.	Protocol-level focus; practical onboarding and fee-free use for non-technical voters is not the main concern.
8	SecVote: Merkle-proof and nullifier based voting with a gasless relayer	Present study (2026)	EVM contract stores only a Merkle root and spent nullifiers; backend relayer pays gas; off-chain ID review and dual-credential OTP; 19 automated tests passed.	Trusts the off-chain identity authority and the relayer; local single-node chain; campus-scale evaluation only.

2.1 Limitations Identified from Literature Survey (Research Gaps)

- **Wallet and fee friction.** Several Ethereum-based designs require each voter to hold an account and pay gas, which is unrealistic for ordinary members of an institution.
- **Cost of on-chain computation.** Schemes that perform cryptographic work or store per-voter data on-chain see costs rise with the number of voters, which limits them to small groups.
- **Trust in a registrar.** Permissioned designs still rely on a trusted party to decide who is eligible, and few describe how that decision is made auditable.
- **Linking identity to ballot.** Systems that expose wallet addresses or require identity at submission time risk linking a voter to a ballot.
- **Public transaction visibility.** In a purely peer-to-peer configuration, a pending transaction containing a proof and nullifier is visible to others before it is mined.
- **Limited evidence.** Many prototype-level proposals describe an architecture but give little reproducible test or cost evidence.

2.2 Research Objectives

- To verify voter eligibility with Merkle proofs against a root stored on an EVM smart contract, without publishing any personally identifiable information on-chain.
- To guarantee that no eligible voter can vote more than once, using a single-use nullifier registry enforced by the contract.
- To remove cryptocurrency and wallet requirements for voters by routing ballots through an automated, gas-paying relayer.
- To support several concurrent elections, each isolated with its own candidates and time window, through a factory contract.
- To establish voter identity off-chain through ID-proof review and a second credential delivered as a six-digit e-mail one-time password.
- To record every significant operational event in an audit log so that the election process can be reviewed afterwards.
- To evaluate the system with automated tests, on-chain cost analysis and mock campus elections, and to report its limitations honestly.

2.3 Product Backlog (Key User Stories with Desired Outcomes)

The product backlog was maintained on a GitHub project board. Each story was given a MoSCoW priority (Must, Should or Could have), functional and non-functional requirements, and acceptance criteria that had to be met before the story was marked done. Table 2.2 lists the stories and the sprint in which each was delivered.

Table 2.2 Product backlog

S.No	User story	Priority	Sprint
US #1	As an election administrator, I want voter eligibility to be proven with Merkle proofs so that no voter data is published on-chain.	Must	1

S.No	User story	Priority	Sprint
US #2	As an election administrator, I want each voter's nullifier to be recorded when used so that nobody can vote twice.	Must	1
US #3	As an administrator, I want a factory contract that deploys isolated elections so that several elections can run at the same time.	Must	1
US #4	As an administrator, I want election, voter and event data stored in a relational database with an audit log so that the process can be reviewed.	Must	2
US #5	As an administrator, I want a dashboard to create elections, upload eligible voters and set an election to active, paused or closed.	Must	2
US #6	As a voter, I want to vote from a browser without a crypto wallet or gas fees so that voting is simple.	Must	2
US #7	As a voter applicant, I want to apply with a PDF ID proof, and as an administrator I want to review it so that only genuine members are enrolled.	Should	3
US #8	As a voter, I want a six-digit OTP sent to my e-mail, separate from my voting secret, so that a stolen secret alone cannot cast a vote.	Must	3
US #9	As a project team, we want the backend to recover from blockchain restarts and the whole flow to be covered by automated tests and mock elections.	Should	3

Figure 2.1 GitHub project board for the SecVote backlog

2.4 Plan of Action (Project Road Map)

The project ran for thirteen weeks, from 31 July to 30 October 2026, and was divided into three sprints. Sprint 1 built the cryptographic and smart-contract core, Sprint 2 added persistence, the relayer API and the user interfaces, and Sprint 3 added identity proofing, e-mail one-time passwords, resilience work and the verification effort. Three project reviews were held with the faculty guide during this period.

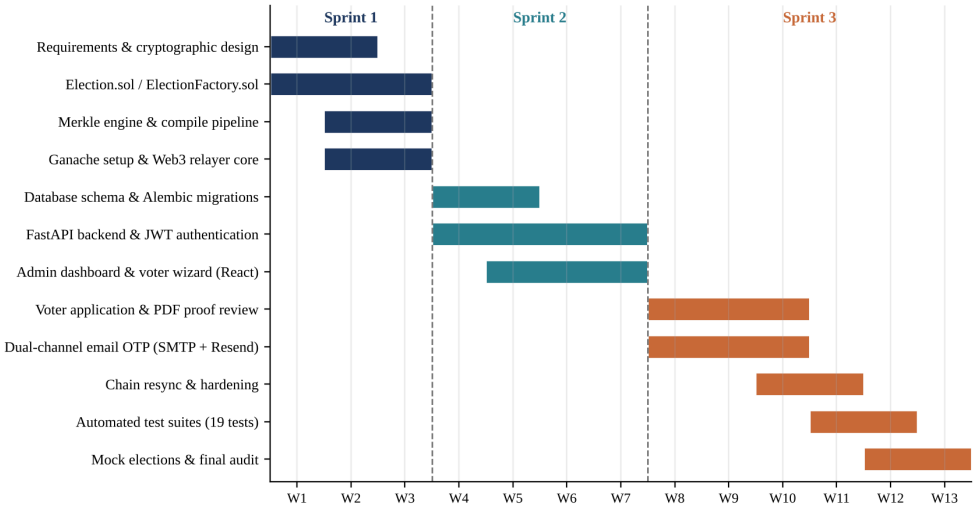


Figure 2.2 Plan of action across the three sprints

CHAPTER 3
SPRINT PLANNING AND EXECUTION METHODOLOGY

3.1 SPRINT 1

3.1.1 Objectives with User Stories of Sprint 1

Sprint 1 ran from 31 July to 21 August 2026 (Weeks 1 to 3). Its goal was to settle the cryptographic design and deliver the on-chain core: smart contracts that accept a ballot only if the voter's Merkle proof is valid and the voter's nullifier is unused, plus the off-chain tree builder and a local test chain to prove that this works.

Table 3.1 Detailed user stories of Sprint 1

S.No	Detailed user story
US #1	As an election administrator, I want voter eligibility to be proven with Merkle proofs so that no voter data is published on-chain. Tasks: define the leaf format, build the tree off-chain, verify proofs on-chain with OpenZeppelin MerkleProof, publish only the root.
US #2	As an election administrator, I want each voter's nullifier to be recorded when used so that nobody can vote twice. Tasks: derive the nullifier with keccak256 from the voter's secret, keep a registry of spent nullifiers, revert on replay.
US #3	As an administrator, I want a factory contract that deploys isolated elections so that several elections can run at the same time. Tasks: write ElectionFactory.sol, give each election its own candidates and time window, export ABIs for the backend.

3.1.2 Functional Document

3.1.2.1 Introduction

Sprint 1 delivers the part of SecVote that does not depend on any user interface: the rules of the election, expressed as Solidity code and a small off-chain library. These rules decide who may vote, how often, and how votes are counted, so they had to be correct before anything else was built on top.

3.1.2.2 Product Goal

The goal of this sprint was to prove, on a local Ethereum-compatible chain, that three properties hold together: a voter on the eligible list can cast a ballot; a person who is not on the list cannot; and nobody can cast a second ballot with the same secret. All three had to hold while storing only a Merkle root and a set of spent nullifiers on-chain.

3.1.2.3 Demography (Users, Location)

- **Users:** election administrators who deploy elections and publish voter roots, and voters whose ballots are submitted on their behalf. In this sprint the users were the developers, interacting through scripts.
- **Location:** institutional elections such as student council, faculty senate and club convenor polls, run on a campus network.

3.1.2.4 Business Processes

- Deploying an election: the factory contract creates an Election instance with its candidates and time window.
- Committing to the voter list: the off-chain tree builder hashes each voter's secret into a leaf, builds the tree and publishes the root with updateMerkleRoot.
- Casting a ballot: a submitter presents the chosen candidate, the nullifier and the Merkle proof; the contract either records the vote or reverts with a reason.

3.1.2.5 Features

- Merkle-root based eligibility check with sorted-pair hashing compatible with OpenZeppelin.
- Nullifier registry that blocks replay of a spent credential.
- Factory-deployed, independent elections with their own time windows.
- Build pipeline that compiles the contracts and exports ABIs for the backend.

3.1.2.6 Authorization Matrix

Table 3.2 Authorization matrix (Sprint 1, contract level)

Role	Access
Election administrator	Deploys elections through the factory; publishes the Merkle root; controls the election window.
Voter	Casts one ballot per valid secret by presenting a Merkle proof; cannot alter tallies or the voter list.
Public auditor	Reads the Merkle root, spent nullifiers, tallies and emitted events; no write access.

3.1.3 Architecture Document

3.1.3.1 Application

- The chain is used as a public bulletin board, not as a database. It holds the Merkle root, the nullifier registry and the tallies, and nothing that identifies a person.
- Heavy work (building the tree, producing proofs) happens off-chain; the chain only verifies. This keeps the cost of publishing the voter list constant.
- Each election is a separate contract instance, which isolates candidates, windows and results from one another.

3.1.3.2 Cryptographic Design

Each voter receives a randomly generated secret of 16 bytes, written as 32 hexadecimal characters. The leaf and the nullifier are both derived from this secret with keccak256, so the list of leaves reveals nothing about the secrets. The leaves are combined pairwise, with each pair sorted before hashing, until a single root remains, which matches the convention of the OpenZeppelin MerkleProof library [9]. To prove membership, the submitter supplies the sibling hashes along the path from the voter's leaf to the root, and the contract recomputes the root and compares it with the stored value. Figure 3.1 illustrates a four-voter tree and the proof for the third voter.

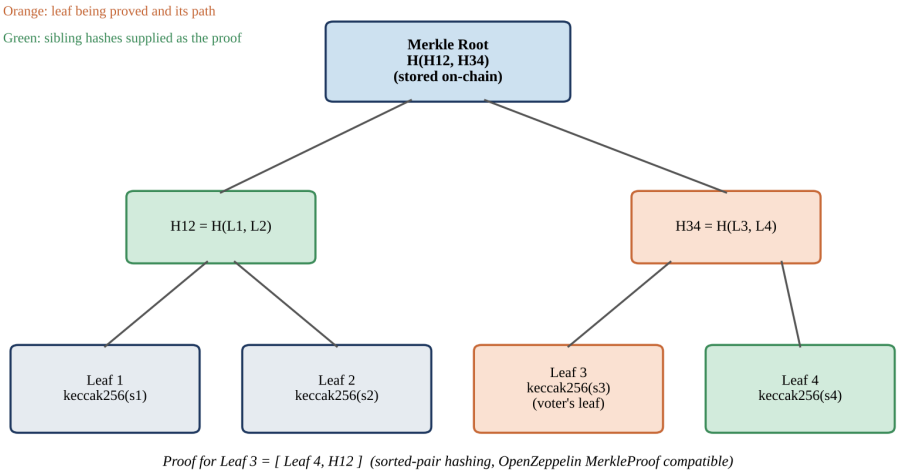


Figure 3.1 Merkle tree and membership proof for one voter

For N eligible voters the proof contains ceil(log2 N) sibling hashes, so a list of 1,000 voters needs ten hashes, about 320 bytes of calldata. Publishing a new root costs the same regardless of how many voters it commits to.

3.1.3.3 Smart-Contract Design

The contracts are written in Solidity ^0.8.24, targeting the paris EVM version with the optimizer set to 200 runs, which favours cheap repeated execution of the ballot function over a small deployment size. Election.sol stores the Merkle root, the registry of spent nullifiers and a tally for each candidate. Its castVote function checks the proof against the root, requires that the nullifier has not been used, marks it used and increments the chosen candidate's tally. An ineligible proof reverts with the message “Election: not on the eligible voter list”. ElectionFactory.sol deploys Election instances and keeps the system multi-tenant. A compile script produces the ABIs that the backend later loads.

3.1.3.4 Local Development Chain

A local Ganache node (JSON-RPC at 127.0.0.1:8545) was used for development. A scripted scenario, fullFlow.js, deploys the contracts, builds a tree, casts valid ballots and then deliberately attempts a replay and an ineligible vote to confirm that both are rejected.

3.1.4 Outcome of Objectives / Result Analysis

The contract test script behaved as designed. Three ballots from eligible voters were accepted; a replayed nullifier was rejected by the registry; and a fabricated leaf with no valid path was rejected with the eligibility error above. Because the root is fixed on-chain and every spent nullifier is recorded, these outcomes do not depend on the honesty of the submitter.

3.1.5 Sprint Retrospective

[Edit this table to reflect what your team actually discussed in the sprint retrospective.]

Table 3.3 Sprint 1 retrospective

What went well	What went poorly	Ideas for next sprint
The Merkle and nullifier design was settled early and verified by script. Using OpenZeppelin MerkleProof avoided writing custom verification.	Contract behaviour could only be inspected through scripts; there was no user interface yet. On-chain cost figures were not yet recorded in an organised way.	Add a database and API so that voter lists and results are not handled by hand. Build a relayer so voters never touch gas.

3.2 SPRINT 2

3.2.1 Objectives with User Stories of Sprint 2

Sprint 2 ran from 22 August to 18 September 2026 (Weeks 4 to 7). Its goal was to turn the contract core into a working platform: a relational database with an audit trail, a FastAPI backend that builds proofs and relays transactions, and React interfaces for administrators and voters.

Table 3.4 Detailed user stories of Sprint 2

S.No	Detailed user story
US #4	As an administrator, I want election, voter and event data stored in a relational database with an audit log so that the process can be reviewed. Tasks: model the elections, voters, otp_sessions, merkle_leaves and audit_log tables; manage the schema with Alembic.
US #5	As an administrator, I want a dashboard to create elections, upload eligible voters and set an election to active, paused or closed. Tasks: role-based login with JWT, election creation form, voter list upload, status controls.
US #6	As a voter, I want to vote from a browser without a crypto wallet or gas fees so that voting is simple. Tasks: voter wizard in React, backend endpoint that builds the proof and relays castVote, result and receipt display.

3.2.2 Functional Document

3.2.2.1 Introduction

With the contracts in place, Sprint 2 had to make them usable by people who are neither developers nor blockchain users. This sprint therefore concentrates on the application layer that sits between a browser and the chain.

3.2.2.2 Product Goal

The product goal was a complete administrator-to-voter path: an administrator creates an election, loads the eligible voters and opens the poll, and a voter then completes the ballot in a few steps, with every action stored in a durable log.

3.2.2.3 Demography (Users, Location)

- **Users:** election administrators (election staff or faculty coordinators) and voters (students or members of the institution) with no blockchain experience.
- **Location:** campus deployments accessed from laptops and phones through a web browser.

3.2.2.4 Business Processes

- Election set-up: the administrator defines candidates and the voting window, uploads the eligible list, and the backend builds the Merkle tree and publishes the root.
- Voting: the voter completes the wizard; the backend builds the proof, the relayer submits the transaction and the voter sees the outcome.
- Supervision: the administrator can pause or close an election, and every state change is written to the audit log.

3.2.2.5 Features

- Relational schema with five core tables and versioned migrations.
- JWT-based authentication with separate administrator and voter roles.
- Administrator dashboard for election creation, voter upload and status control (Active, Paused, Closed).
- Voter wizard that hides all blockchain details.
- Gasless relaying: the backend pays all transaction fees.

3.2.2.6 Authorization Matrix

Table 3.5 Authorization matrix (Sprint 2, application level)

Role	Access
Administrator	Create elections, upload eligible voters, change election status, view audit log and results.
Voter	Authenticate and cast one ballot in an active election; view own confirmation.
Unauthenticated visitor	Reach the login page only; all other API routes reject the request.

3.2.2.7 Assumptions

- A local Ganache node is running and funded accounts exist for the relayer.
- The eligible voter list supplied by the administrator is accurate.
- The system runs on a trusted institutional server, since the relayer holds the key that pays for transactions.

3.2.3 Architecture Document

3.2.3.1 Application

- The three-layer structure (interface, application, persistence plus chain) keeps cryptography on the server and keeps the browser simple.
- The backend is the only component that talks to the blockchain, so voters never need a wallet and the node is not exposed to the internet.
- Database migrations make the schema reproducible on another machine.

3.2.3.2 Architecture Diagram

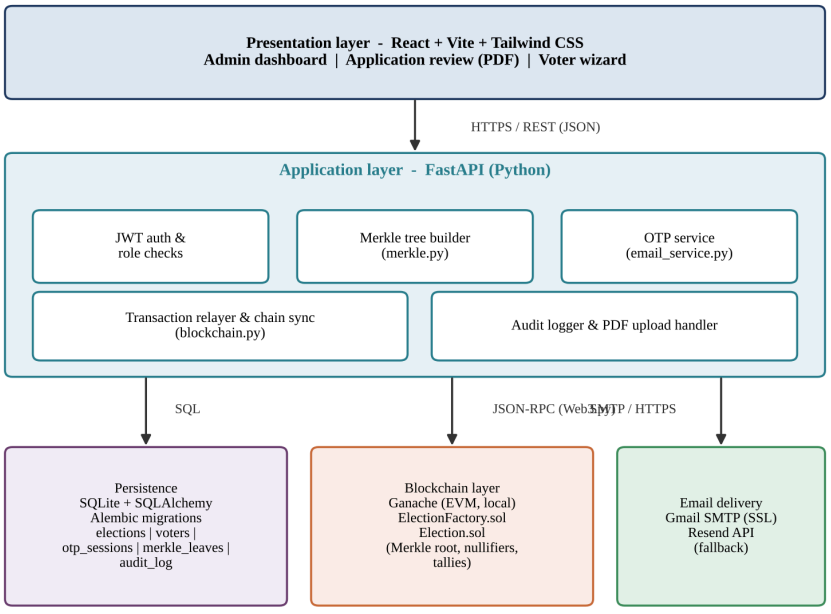


Figure 3.3 SecVote system architecture

3.2.3.3 Data Model

Table 3.6 Core database entities

Table	Purpose
elections	Election metadata, candidates, time window, status and the address of the deployed contract.
voters	Enrolled voters and their credentials state for an election.
otp_sessions	Pending one-time passwords: hashed code, expiry and attempt counter.
merkle_leaves	Leaf hashes used to rebuild the tree and generate proofs.
audit_log	Time-stamped events (OTP requests, proof generation, relay results) with voter references and transaction hashes.

3.2.3.4 Backend and Relayer

The backend is written in Python with FastAPI [11] and served with Uvicorn. SQLAlchemy provides the ORM and Alembic manages migrations. Web3.py connects to the node, and a relayer account signs and pays for every castVote transaction. Because the relayer is the transaction sender, a voter's own address never appears on the chain, and the voter pays nothing.

3.2.4 Outcome of Objectives / Result Analysis

By the end of the sprint an administrator could create an election, upload a voter list, publish its root and change its status, and a voter could complete the wizard without any wallet. The audit log recorded the operational events of the test runs. The remaining weakness was identity: at this stage anyone holding a valid secret could vote, which motivated the work of Sprint 3.Figure 3.4 Administrator dashboardxFigure 3.5 Voter voting wizard

3.2.5 Sprint Retrospective

[Edit this table to reflect what your team actually discussed in the sprint retrospective.]

Table 3.7 Sprint 2 retrospective

What went well	What went poorly	Ideas for next sprint
The relayer removed all wallet friction for voters. The audit log gave visibility into every step of a run.	Anyone holding a valid secret could vote; identity was not yet checked. Test coverage of the backend was thin.	Add a voter application with ID proof and administrator review. Add a second credential (OTP) and automated tests.

3.3 SPRINT 3

3.3.1 Objectives with User Stories of Sprint 3

Sprint 3 ran from 19 September to 30 October 2026 (Weeks 8 to 13). Its goal was to make the system trustworthy enough to run real elections: proof of identity, a second credential, resilience to infrastructure failures, and systematic verification.

Table 3.8 Detailed user stories of Sprint 3

S.No	Detailed user story
US #7	As a voter applicant, I want to apply with a PDF ID proof, and as an administrator I want to review it so that only genuine members are enrolled. Tasks: application form with PDF upload, review screen, approve/reject, voter password.
US #8	As a voter, I want a six-digit OTP sent to my e-mail, separate from my voting secret, so that a stolen secret alone cannot cast a vote. Tasks: OTP generation and hashing, five-minute expiry, attempt limit, Gmail SMTP with Resend fallback.
US #9	As a project team, we want the backend to recover from blockchain restarts and the whole flow to be covered by automated tests and mock elections. Tasks: ensure_chain_sync(), OTP and end-to-end test suites, mock elections, final audit.

3.3.2 Functional Document

3.3.2.1 Introduction

Sprint 3 closes the gap left by Sprint 2. The platform could already count ballots correctly; it now had to make sure that the person holding a voting secret was also the person entitled to it.

3.3.2.2 Product Goal

The goal was a two-factor voting flow in which a ballot needs both something the voter was issued (the secret key, Credential A) and something delivered to the voter's e-mail at voting time (the one-time password, Credential B), backed by a documented review of each applicant's identity.

3.3.2.3 Demography (Users, Location)

- **Users:** applicants who become voters after approval, and administrators who review applications.
- **Location:** campus elections where members hold institutional e-mail addresses.

3.3.2.4 Business Processes

- Enrolment: an applicant submits details and an ID-proof PDF; an administrator reviews and approves or rejects.
- Credential issue: an approved voter receives a secret key, and the eligible list is committed to the Merkle root.
- Voting: the voter logs in, requests an OTP, and submits the ballot with the secret and the OTP.
- Recovery: if the local chain is restarted, the backend detects the mismatch and re-synchronises.

3.3.2.5 Features

- Voter application with PDF proof upload and administrator review.
- Voter password, stored as a hash, added to the account model.
- Six-digit OTP with a five-minute lifetime and at most five invalid attempts before the session is locked.
- Dual-channel e-mail delivery: Gmail SMTP over SSL (port 465) with the Resend API as a fallback.
- Self-healing chain synchronisation (ensure_chain_sync) after node restarts.

3.3.2.6 Authorization Matrix

Table 3.9 Authorization matrix (Sprint 3)

Role	Access
Applicant	Submit an application and ID-proof PDF; check application status.
Voter (approved)	Log in, request an OTP and cast one ballot with the secret key and OTP.
Administrator	Review uploaded proofs, approve or reject applications, manage elections and view the audit log.
Public auditor	Verify tallies, nullifiers and the Merkle root directly on the contract.

3.3.3 Architecture Document

3.3.3.1 Application

- The two credentials are deliberately independent: the secret is issued once at enrolment, while the OTP is created at voting time and expires, so compromising one does not give the other.
- Two mail channels were used because delivery to college addresses proved unreliable through a single provider.
- Every step of the flow writes to the audit log, which becomes the evidence base for the results in Chapter 4.

3.3.3.2 Enrolment and Voting Flow

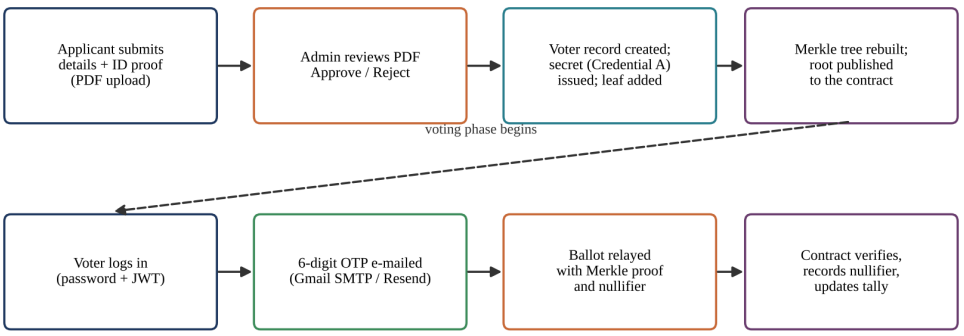


Figure 3.6 Voter enrolment and voting workflow

3.3.3.3 Ballot Sequence

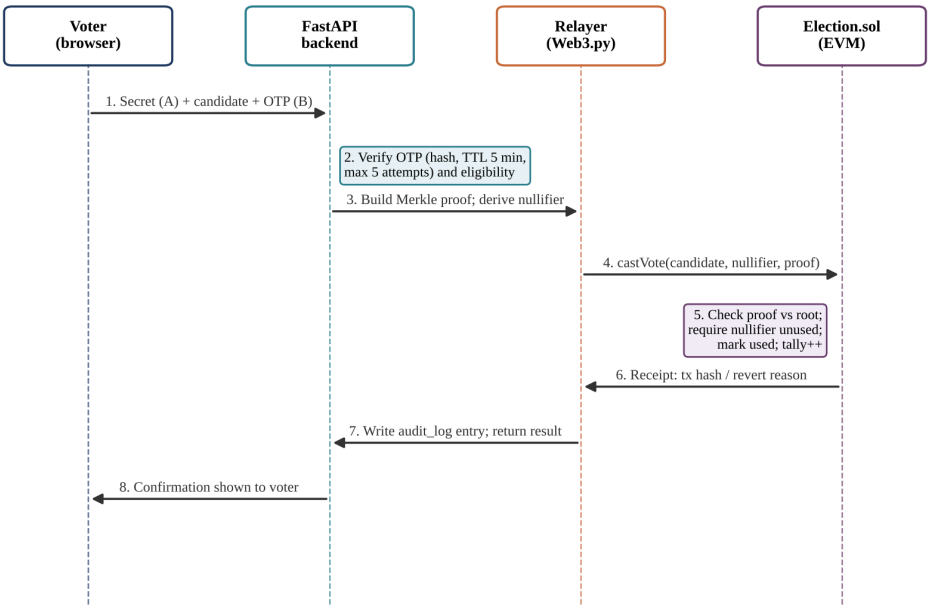


Figure 3.7 Sequence of a ballot from voter to contract

3.3.3.4 One-Time Password Design

An OTP is a random six-digit number, giving one million possible values. Only a SHA-256 hash of the code is stored in the otp_sessions table, the code expires after five minutes, and the session locks after five invalid attempts, so an attacker cannot guess the code by brute force within its lifetime. The OTP is checked by the backend before any transaction is built, so a wrong OTP never reaches the chain or costs gas.

3.3.3.5 Resilient Chain Synchronisation

A development node such as Ganache loses its state when it is restarted, which would leave the backend pointing at contract addresses that no longer exist. The ensure_chain_sync routine checks that the recorded contracts are present on the connected node and re-establishes them when they are not, so that a restart does not require manual repair.

3.3.4 Outcome of Objectives / Result Analysis

All three stories were completed. Sixteen identity-proof PDFs were uploaded during testing and handled through the administrator review screen. OTP e-mails were

delivered through the Gmail SMTP channel, with the Resend API available as a fallback. Two live mock elections, named DISCIPLINE CONVENORS and SRM Student Council Election 2026, were run on campus using the full flow. The automated suites described in Chapter 4 passed in full: 19 of 19 tests. 3.9 Automated test run for the OTP and end-to-end suites

3.3.5 Sprint Retrospective

[Edit this table to reflect what your team actually discussed in the sprint retrospective.]

Table 3.10 Sprint 3 retrospective

What went well	What went poorly	Ideas for the future
Two-factor voting and ID review made the flow credible for real elections. All 19 automated tests passed and the mock elections ran end to end.	College e-mail deliverability was unreliable through one provider. Restarting the local chain broke the backend until synchronisation logic was added.	Replace the trusted secret issuer with zero-knowledge membership proofs. Move from a single local node to a permissioned or layer-2 network.

CHAPTER 4
RESULTS AND DISCUSSIONS

4.1 Project Outcomes (Performance Evaluation, Comparisons, Testing Results)

SecVote is a cryptographic and distributed-systems project, so its behaviour is deterministic: a ballot is either accepted or rejected according to rules enforced by code. For that reason the evaluation uses verification tests, on-chain cost analysis and operational evidence rather than statistical measures such as accuracy on a held-out dataset.

4.1.1 Verification Results

Table 4.1 Security and verification results

Property	Result	How it was checked
Eligibility verification	Valid Merkle proofs always accepted; a fabricated leaf rejected with “Election: not on the eligible voter list”	Contract test script fullFlow.js
Double-voting prevention	A replayed nullifier was rejected by the spent-nullifier registry	Contract test script fullFlow.js
Credential separation	The voting secret (Credential A) and the e-mail OTP (Credential B) are validated independently	test_otp_workflow.py
Audit-trail completeness	338 events recorded with timestamps, event types, voter references and transaction hashes	audit_log table in secvote.db
Automated test pass rate	19 of 19 tests passed	Three test suites (Table 4.2)

Table 4.2 summarises the outcome of the three automated suites, and Table 4.3 gives the verification matrix for the ballots submitted in the contract test scenario.

Table 4.2 Automated test suites

Suite	File	Tests	Result
Smart-contract security	voting-contracts/test/fullFlow.js	5	5 passed
OTP security and lockout	voting-backend/test_otp_workflow.py	6	6 passed
End-to-end workflow and audit	voting-backend/test_e2e_flow.py	8	8 passed
Total		**19**	**19 passed (100%)**

Table 4.3 Verification matrix for the contract test scenario

Actual voter status	Accepted	Rejected
Eligible voter, first attempt	3	0
Ineligible (fabricated) voter	0	1
Replay of a spent nullifier	0	1

In this scenario every eligible first attempt was accepted and every illegitimate attempt was rejected, with no false acceptances. The sample is small and scripted, so it demonstrates that the rules work as designed rather than estimating a rate over a large population.

4.1.2 On-Chain Cost

Table 4.4 lists the gas measured for the main contract operations on the local chain. The cost of publishing a Merkle root does not depend on the number of voters, while the cost of a ballot depends on the length of the proof. The first vote in an election costs more than later votes because it performs first-time storage writes.

Table 4.4 Measured gas consumption of contract operations

Operation	Gas used	Scaling
createElection	1,193,696	Per election
updateMerkleRoot	32,220	Constant, independent of voter count
castVote (first vote in an election)	102,219	Includes first-time storage writes
castVote (later votes)	85,097	Grows with proof length, O(log N)

The value of committing to the voter list with a root can be seen by comparison. Writing a new storage slot on the EVM costs on the order of 20,000 gas or more, so recording each of 1,000 voters on-chain would cost at least 20 million gas.

4.1.3 Proof Size and Scalability

A Merkle proof contains ceil(log2 N) sibling hashes of 32 bytes each. Table 4.5 and Figure 4.1 show how slowly this grows: raising the electorate from one thousand to one million voters adds only ten hashes, about 320 bytes of calldata, to each ballot.

Table 4.5 Merkle proof length by electorate size

Eligible voters N	Proof length (hashes)	Proof size (bytes)
100	7	224
1,000	10	320
10,000	14	448
100,000	17	544
1,000,000	20	640

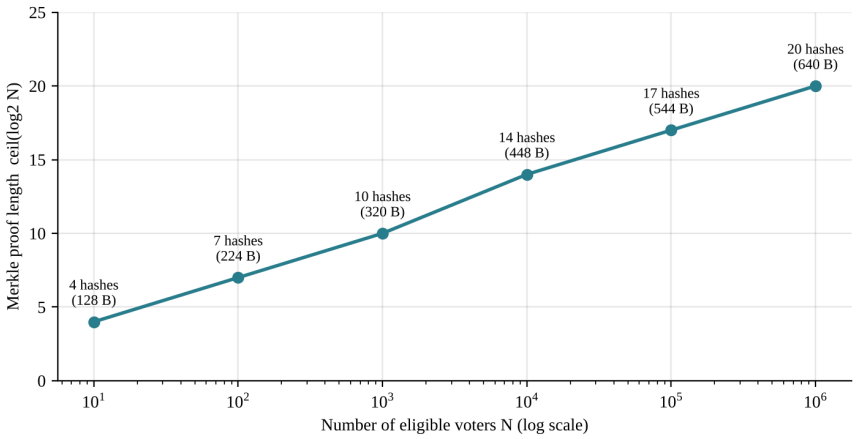


Figure 4.1 Merkle proof length as the electorate grows

These proof sizes are computed from the tree structure. Off-chain tree construction was timed on an Apple M1 laptop with 8 GB of memory: 14.23 ms for 1,000 voters and 165.26 ms for 10,000 voters. The system was exercised at campus scale, and no end-to-end load test with thousands of simultaneous voters was carried out.

4.1.4 Operational Evidence

Table 4.6 Operational data gathered during testing

Item	Value
Audit-log events recorded in the database	338
Identity-proof PDFs uploaded for review	16
Live mock elections run	2 (DISCIPLINE CONVENORS; SRM Student Council Election 2026)
OTP parameters	6 digits, 5-minute lifetime, 5 invalid attempts before lockout

4.2 Threat Analysis

Table 4.7 Threats considered and residual risks

Threat	Mitigation in SecVote	Residual risk
Voting twice	Spent-nullifier registry on the contract	None within the contract; depends on issuing one secret per person.
Ineligible person votes	Merkle proof against the published root; ID-proof review	Administrator approves a wrong applicant.
Stolen voting secret	Second credential: e-mail OTP	Attacker also controls the voter's mailbox.
Guessing the OTP	Hashed storage, 5-minute lifetime, 5-attempt lockout	Low.
Observing a pending ballot	Backend submits to a private local node as the transaction sender	On a public chain the transaction would be visible in the mempool; private submission or commit-reveal would be needed.
Linking a voter to a ballot	No personal data on-chain; relayer is the sender	The backend and relayer see which candidate an authenticated voter chose and could record it if malicious or compromised.
Minting rogue secrets	Secrets issued only after administrator approval	A compromised administrator server could issue illegitimate secrets.

4.2.2 Limitations

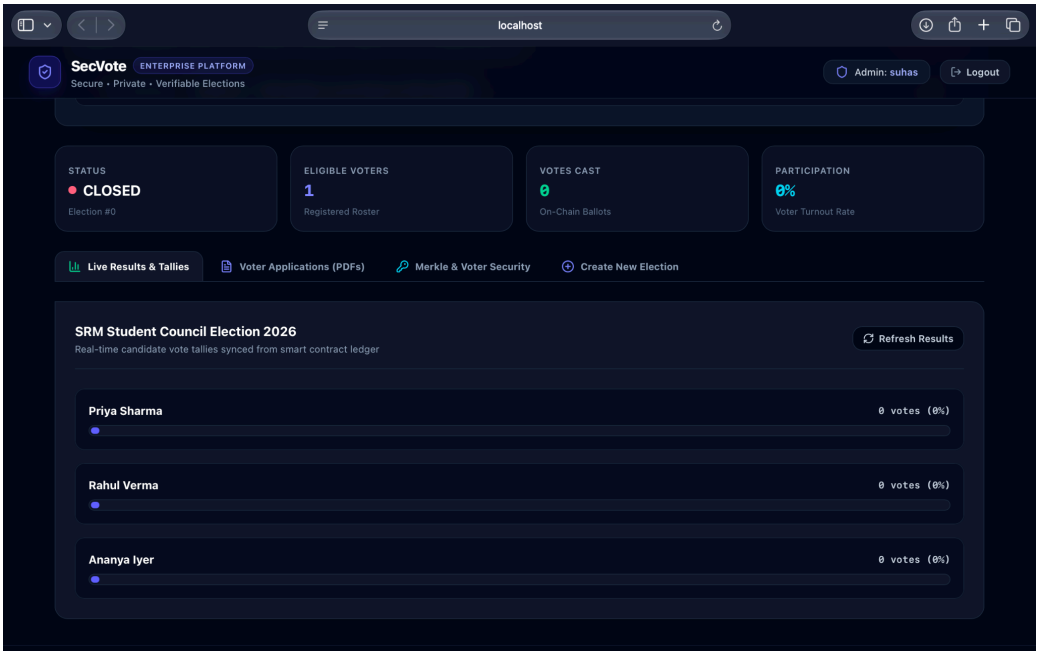
- **Trusted identity authority.** The mapping from a real identity to a secret is handled by the election backend. Ballot secrecy holds against observers of the chain, but not against a malicious or compromised backend, which knows who authenticated.
- **Trusted relayer.** The relayer sees the link between each authenticated voter and the chosen candidate, and it could in principle refuse to submit a ballot. It is a single gas-paying wallet, so if that wallet ran out of funds, voting would stop. The system does not yet let a voter submit independently.
- **Single local node.** The evaluation used one Ganache node, so there was no decentralised consensus. A production election would need a permissioned or public network.

- **No receipt-freeness or coercion-resistance proof.** A voter holding a secret and a transaction hash could be pressured to show how they voted; the design does not claim protection against this.
- **Manual identity review and no national eKYC.** ID proofs are reviewed by an administrator. Government eKYC services need organisational certification and security hardware that a student project does not have.
- **Prototype infrastructure.** SQLite and a local chain are suitable for a demonstration, and a larger deployment would need a production database and load testing.

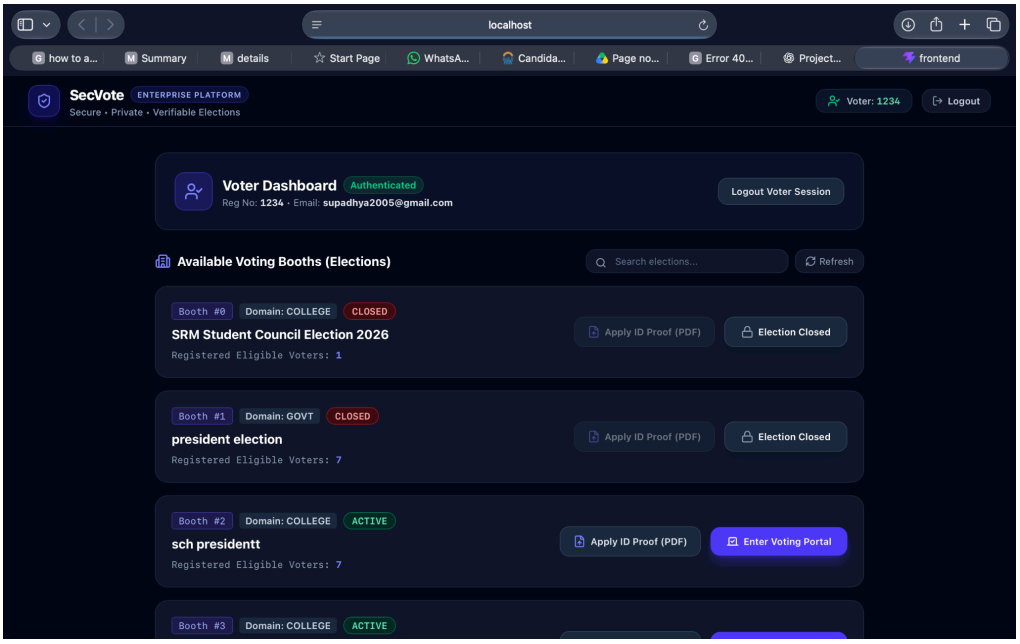
4.2.3 Comparison with Related Work

Compared with the boardroom-voting approach of McCorry et al. [4], SecVote moves cryptographic work off-chain and lets the contract only verify a short proof, which suits larger electorates and non-technical voters. Compared with the permissioned-ledger designs surveyed in [5] and [6], it keeps the EVM smart-contract ecosystem and its tooling, while adding the identity review and second credential that those works treat as outside their scope. It does not offer the stronger privacy and coercion properties of the more elaborate protocols, and Chapter 5 outlines how zero-knowledge techniques could narrow that gap.

ADMIN DASHBOARD



VOTER DASHBOARD



CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

This project set out to build a voting system that keeps voter identity off a public ledger, prevents double voting with a mathematical guarantee, remains publicly auditable and does not ask voters to handle cryptocurrency. SecVote achieves this with a small set of mechanisms: a Merkle root committing to the eligible list, single-use nullifiers recorded on an EVM contract, a gas-paying relayer, and an off-chain layer of ID review and two credentials.

The evidence supports the central claims. In the contract test scenario, valid ballots were accepted while a fabricated voter and a replayed nullifier were rejected. All 19 automated tests passed. The audit log captured 338 operational events, and the system was used in two mock elections. The cost analysis showed that publishing the voter list costs a constant amount, and that each ballot grows only logarithmically with the electorate.

The work also showed where the limits lie. Ballot secrecy is protected against chain observers but still depends on trusting the backend that issues secrets and relays ballots. These are properties of the current design, and the next section describes how they could be reduced.

5.2 Future Enhancement

- **Zero-knowledge membership proofs.** Replacing plain Merkle proofs with zero-knowledge proofs of membership would let a voter prove eligibility without revealing which leaf is theirs, removing the backend's ability to link a voter to a ballot.
- **Anonymous credentials or blind issuance.** Issuing voting credentials with blind signatures would prevent even the issuer from knowing which secret belongs to which voter.
- **Decentralised relaying.** Supporting several independent relayers, or meta-transactions that a voter can submit through any relayer, would remove the single point of censorship.

- **Production-grade network.** Deploying on a permissioned multi-validator network or a public layer-2 would give real consensus, while hardware security modules would protect the relayer's signing key.
- **Stronger identity verification.** Integrating a certified eKYC service or institutional single sign-on would reduce reliance on manual PDF review.
- **Assurance work.** Static analysis and fuzzing of the contracts, an independent security review, a production database and large-scale load tests would be needed before use in a binding election.
- **Coercion resistance and receipt-freeness.** Researching re-voting or similar mechanisms would address pressure on voters to prove their choice.

REFERENCES

[1]J. Benaloh, “Verifiable secret-ballot elections,” Ph.D. dissertation, Dept. of Computer Science, Yale University, New Haven, CT, USA, 1987.

[2]R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology - CRYPTO '87*, Lecture Notes in Computer Science, vol. 293. Berlin, Germany: Springer, 1988, pp. 369-378.

[3]N. Kshetri and J. Voas, “Blockchain-enabled e-voting,” *IEEE Software*, vol. 35, no. 4, pp. 95-99, Jul./Aug. 2018, doi: 10.1109/MS.2018.2801546.

[4]P. McCorry, S. F. Shahandashti, and F. Hao, “A smart contract for boardroom voting with maximum voter privacy,” in *Financial Cryptography and Data Security*, Lecture Notes in Computer Science, vol. 10322. Springer, 2017, pp. 357-375.

[5]F. Þ. Hjálmarsson, G. K. Hreiðarsson, M. Hamdaqa, and G. Hjálmtýsson, “Blockchain-based e-voting system,” in *Proc. IEEE 11th Int. Conf. Cloud Computing (CLOUD)*, 2018, pp. 983-986, doi: 10.1109/CLOUD.2018.00151.

[6]S. Bistarelli, I. Mercanti, P. Santancini, and F. Santini, “End-to-end voting with non-permissioned and permissioned ledgers,” *Journal of Grid Computing*, vol. 17, no. 1, pp. 97-118, 2019, doi: 10.1007/s10723-019-09478-y.

[7]F. S. Hardwick, A. Gioulis, R. N. Akram, and K. Markantonakis, “E-voting with blockchain: An e-voting protocol with decentralisation and voter privacy,” in *Proc. IEEE iThings/GreenCom/CPSCom/SmartData*, 2018, arXiv:1805.10258.

[8]G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” Ethereum Yellow Paper.

[9]OpenZeppelin, “Contracts documentation: MerkleProof,” docs.openzeppelin.com/contracts.

[10]United Nations General Assembly, “Transforming our world: the 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, 2015.

[11]S. Ramírez, “FastAPI documentation,” fastapi.tiangolo.com.

APPENDIX A
CODING

Table A.1 lists the key code artifacts of the SecVote repository. Screenshots of the most relevant code should be inserted below it.

Table A.1 Key code artifacts

File	Role
voting-contracts/contracts/Election.sol	Core election contract: Merkle root, spent-nullifier registry, castVote, tallies.
voting-contracts/contracts/ElectionFactory.sol	Factory that deploys isolated Election instances.
voting-contracts/test/fullFlow.js	Contract security test scenario (5 tests).
voting-backend/app/merkle.py	Off-chain binary Merkle tree with sorted-pair hashing.
voting-backend/app/blockchain.py	Web3 connection layer and ensure_chain_sync().
voting-backend/app/email_service.py	Dual-channel OTP delivery (Gmail SMTP and Resend).
voting-backend/test_otp_workflow.py	OTP security and lockout tests (6 tests).
voting-backend/test_e2e_flow.py	End-to-end workflow and audit tests (8 tests).
frontend/src/components/AdminView.jsx	Administrator dashboard and application review.
frontend/src/components/VoterView.jsx	Voter wizard.
frontend/src/components/LoginPage.jsx	Role-based login page.